UNIT -IV

GSM

GSM GPRS Modules are one of the commonly used communication modules in embedded systems. A GSM GPRS Module is used to enable communication between a microcontroller (or a microprocessor) and the GSM / GPSR Network. Here, GSM stands for Global System for Mobile Communication and GPRS stands for General Packet Radio Service. A GSM GPRS MODEM comprises of a GSM GPRS Module along with some other components like communication interface (like Serial Communication – RS-232), power supply and some indicators. With the help of this communication interface, we can connect the GSM GPRS Module on the GSM GPRS MODEM with an external computer (or a microcontroller).

GPRS or General Packet Radio Service is an extension of the GSM Network. GPRS is an integrated part of the GSM Network which provides an efficient way to transfer data with the same resources as GSM Network.

Originally, the data services (like internet, multimedia messaging etc.) in the GSM Network used a circuit – switched connection. In this type, the access time for the network are long and the charges for the data were based on the connection time. Also, this type of connection is not suitable for transmitting bursts of data.

With the integration of GPRS, a packet – switching based data service, in to the GSM Network, the scene of data services has changes. In GPRS based packet – switching networks, the user device doesn't hold the resources for a continuous time but efficiently uses a common pool.

The access time in GPRS is very small and the main advantage is that it allows bursts of data to be transmitted. Also, the charges for data are based on the usage and not on the connection time.

The Working of a GSM Network

GSM is a digital cellular communication standard that is universally accepted. The European Telecommunications Standards Institute created the GSM standard to define the procedures for second-generation digital mobile networks that are used by devices such as mobile phones. It is a wide-area communications technology program that utilizes digital radio channeling to bring forth audio, information, and multimedia communication systems.

GSM is a mobile network and not a computer networkOpens a new window – this implies that devices interact with it by looking for nearby cells. GSM, including other technological advances, has influenced the evolution of mobile wireless telecommunication services. A GSM system manages communication between mobile stations, base stations, and switching systems.

Every GSM radio channel is 200 kHz wide and is additionally divided into frames of 8-time slots. The global system for mobile communication (GSM) was first known as Groupe Special Mobile, which is the reason for the acronym. The GSM system comprises mobile stations, base stations, and intertwining switching systems.

The GSM program enables 8 to 16 audio users to share every radio channel, and every radio transmission location may have multiple radio channels. Because of its simplicity, affordability, and accessibility, GSM is presently the most commonly used network technology in the Internet of Things (IoT)Opens a new window applications.

However, this is likely to change in the coming years. Various programs have been designed without the advantage of standardized provisions all through the transformation of mobile telecommunication services.

This significantly created many issues tied directly to consistency as digital radio technology advanced. The global system for mobile communication is designed to address these issues. GSM accounts for about 70% of the world's digital cellular services. GSM automates and encodes the information before transmitting it via a channel including three distinct streams of user information inside each time slot. For the vast majority of the world, it is also the leading 2G digital cell phone standard. It governs how cell phones interact with the land-based tower system.

GSM-SIM900A

The GSM-SIM900A modem is a GSM/GPRS module based on the SIM900A chip from SIMCom. It allows devices to connect to GSM networks for sending and receiving voice, SMS, and data over GPRS. The working principle of the GSM-SIM900A modem involves several key steps and components, which are outlined below:

1. Hardware Components

- **SIM900A Module**: The core component, which handles all GSM/GPRS communication.
- **SIM Card Slot**: Holds the SIM card that identifies the subscriber on the GSM network.
- **Antenna**: Ensures good signal reception for GSM communication.
- **Power Supply**: Provides the necessary voltage and current to operate the module.
- **Serial Interface**: Typically UART (Universal Asynchronous Receiver-Transmitter) for communication with microcontrollers or computers.

2. Initialization

1. **Power On**: The modem is powered on, initializing the SIM900A module.
2. **Network Registration**: The module searches for available GSM networks and attempts to register with the network using the SIM card credentials.

3. AT Commands

The GSM-SIM900A modem communicates with a host device (e.g., a microcontroller or computer) using AT commands (Attention Commands). These commands are used to control the modem and perform various functions such as sending SMS, making calls, and establishing GPRS connections.

4. Basic AT Commands

1. **Checking Connectivity**: AT
    - Sends a simple command to check if the modem is responsive. The modem should reply with OK.
2. **Network Registration Status**: AT+CREG?
    - Checks if the modem is registered on the GSM network.
3. **Signal Quality**: AT+CSQ
    - Returns the signal strength of the network.
4. **SIM Card Status**: AT+CPIN?
    - Checks if the SIM card is inserted and ready.

5. **Sending SMS**: AT+CMGF=1 (sets the SMS mode to text), followed by AT+CMGS="+<recipient_number>" to send the SMS.
    - Allows sending of SMS messages to a specified number.
6. **Making a Call**: ATD<phone_number>;
    - Initiates a voice call to the specified phone number.
7. **GPRS Connection Setup**: Involves multiple commands such as AT+CGATT=1 (attach to GPRS service), AT+SAPBR (configure bearer), and AT+HTTP commands to perform HTTP operations.

5. Communication Flow

1. **Establish Connection**: The host device sends AT commands via the serial interface to establish a connection with the GSM network.
2. **Perform Operations**: Depending on the requirement, the host can perform various operations such as sending SMS, making calls, or using GPRS for data transmission.
3. **Receive Responses**: The modem responds to each command with a result code (e.g., OK, ERROR, or data response).

6. Example Workflow for Sending an SMS

1. **Initialize SMS Text Mode**: AT+CMGF=1
2. **Send SMS Command**: AT+CMGS="+<recipient_number>"
3. **Enter Message Text**: <Your message here> followed by Ctrl+Z (ASCII code 26) to send the message.
4. **Receive Response**: The modem sends back a confirmation indicating the SMS has been sent.

7. Power Management

The SIM900A module supports various power-saving modes to reduce power consumption when the modem is not actively transmitting or receiving data. This is crucial for battery-operated devices.

Summary

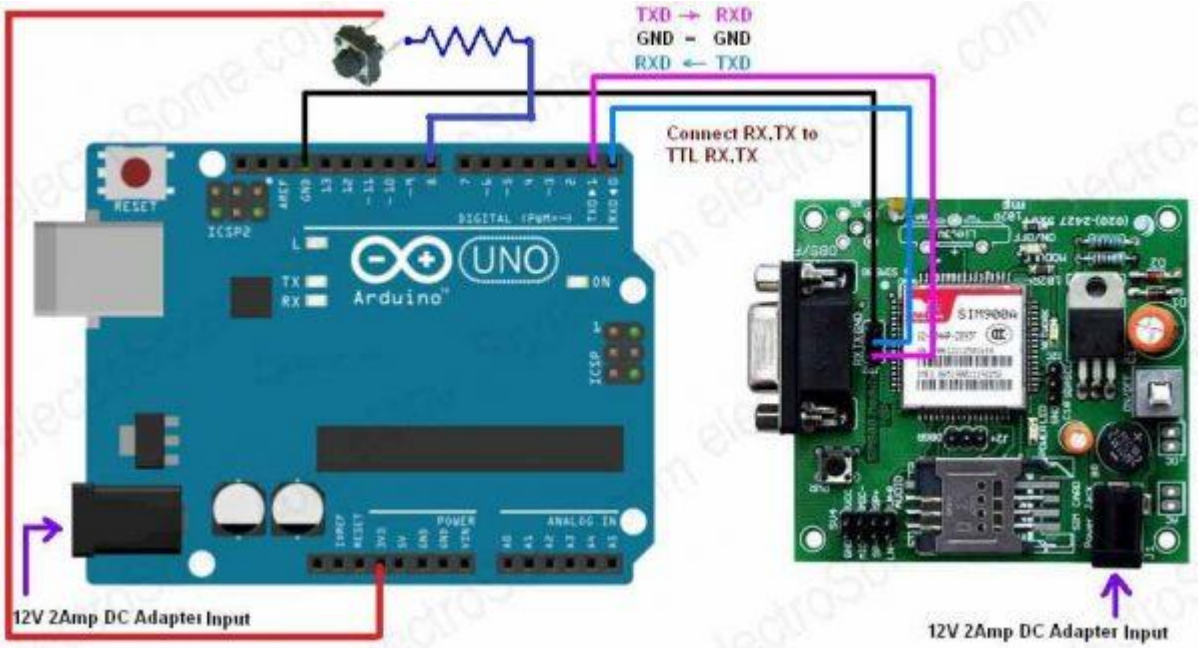
The GSM-SIM900A modem allows devices to connect to GSM networks using the following principles:

- **Hardware Setup**: Includes the SIM900A module, SIM card, antenna, and power supply.
- **Initialization**: Powering on the modem and registering it on the GSM network.
- **AT Commands**: Used for controlling the modem and performing operations like sending SMS, making calls, and setting up GPRS connections.

- **Communication Flow**: Establishing connection, performing operations, and receiving responses.
- **Power Management**: Optimizing power consumption through various power-saving modes.

Understanding these principles allows effective integration and utilization of the GSM-SIM900A modem in various applications such as IoT, remote monitoring, and mobile communication.

Interfacing GSM with Arduino



Connections

- Connect **TX pin of GSM Module to RX pin** of Arduino Uno.
- Connect **RX pin of GSM Module to TX pin** of Arduino Uno.
- Connect GND pin of GSM Module to GND pin of Arduino Uno.

Steps to boot GSM module

1. Power ON the GSM module by providing 5V and GND.
2. Insert the SIM card to GSM module and lock it.

3. Initially blinking rate of network LED will be high. After sometime observe the blinking rate of ‘network LED’ (GSM module will take some time to establish connection with mobile network)
4. Once the connection is established successfully, the network LED will blink continuously for every 3 seconds.
5. Even we can check the connection establishment of GSM module with mobile by making a call to the number of the SIM. If we hear a ring back, the GSM module has successfully established network connection.

Call and Message Program

```
#include <LiquidCrystal.h>
// initialize the library with the numbers of the interface pins
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
const int ledpin =  13;
void setup()
{
  pinMode(7,INPUT);
  pinMode(ledpin,OUTPUT);
  lcd.begin(16, 2);
  Serial.begin(9600);
 lcd.setCursor(0,0);
                    lcd.print(" EXAM CELL ");
                    lcd.setCursor(0,1);
                    lcd.print("SECURITY SYSTEM ");
                    delay (5000);       // Relay on for 5 sec
                    lcd.clear();
}
void callsomeone()
{
  Serial.println("AT");
```

```
  delay(900);

  Serial.println("ATD9994442064;");
  delay(20000);
  Serial.println("ATD");
   Serial.println("AT");
  delay(900);
  Serial.println("ATD9791771361;");
  delay(20000);
  Serial.println("ATD");
  Serial.println("AT");
  delay(900);
  Serial.println("ATD9043289873;");
  delay(20000);
  Serial.println("ATD");
  delay(6000);
 }
void textsomeone()
{
  Serial.println();
  Serial.println("AT+CMGF=1;");
  delay(9000);
  Serial.print("AT+CMGS=\"");
   Serial.print("8344578121");
   Serial.println("\"");
  delay(900);
  Serial.print("Exam cell-Question paper Leaked: ");
  delay(600);
```

```
 Serial.write(byte(26));
}
void loop()
{
  if(digitalRead(7)==HIGH)
  {
    digitalWrite(ledpin, HIGH);
     lcd.setCursor(0,0);
                    lcd.print(" QUESTION PAPER  ");
                    lcd.setCursor(0,1);
                    lcd.print("LEAKED BE ALERT ");
                      delay (5000);       // Relay on for 5 sec
                  callsomeone();
                  //digitalWrite(ledpin, HIGH);
  }
  else
  {
    digitalWrite(ledpin, LOW);
     lcd.setCursor(0,0);
                    lcd.print("EXAM CELL  ");
                    lcd.setCursor(0,1);
                    lcd.print("NORMAL CONDITION");
                      delay (10000);       // Relay on for 5 sec
                    lcd.clear();
  }
}
```

Home Appliance control through GSM

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 10, 9, 8, 7);
char inchar;
void setup()
{
  lcd.begin(16, 2);
 // pinMode(6, OUTPUT);
  pinMode(13, OUTPUT);
   pinMode(3, INPUT);
  digitalWrite(A0,LOW);
    Serial.begin(9600);
  delay(3000); // give time for GSM module to register on network etc.
  Serial.println("AT+CMGF=1"); // set SMS mode to text
  delay(200);
  Serial.println("AT+CNMI=2,2,0,0"); // set module to send SMS data to serial out upon receipt
  delay(200);
  lcd.setCursor(0,0);
  lcd.print("Safe & Security ");
  delay(4000);
  lcd.setCursor(0,0);
  lcd.print("   System for   ");
  lcd.setCursor(0,1);
  lcd.print("  TWO WHEELER   ");
  delay(1000);
  lcd.setCursor(0,0);
  lcd.print("                ");
```

```
  lcd.setCursor(0,1);
  lcd.print("                ");
  lcd.setCursor(0,0);
  lcd.print("Enter Password  ");
  lcd.setCursor(0,1);
  lcd.print("through GSM     ");
  delay(2000);
  digitalWrite(6, LOW);
}
void loop()
{
  int push1 = digitalRead(3);
   // print out the value you read:
  lcd.setCursor(0,0);
  lcd.print("                ");
  lcd.setCursor(0,1);
  lcd.print("                ");

  if (push1 == HIGH)
  {
    check1();
  }
}
void check1()
{
  if(Serial.available() >0)
  {
    inchar=Serial.read();
    if (inchar=='#')
```

```
    {
      delay(10);
      inchar=Serial.read();

      //first led
      if (inchar=='1')
      {
        delay(10);
        inchar=Serial.read();

        if (inchar=='2')
        {
          lcd.setCursor(0,0);
          lcd.print("PASSWORD OK     ");
          lcd.setCursor(0,1);
          lcd.print("****************");
          digitalWrite(A0,HIGH);
          delay(2000);
        }
        delay(10);
      }
      Serial.println("AT+CMGD=1,4"); // delete all SMS
    }
  }
}
```

Introduction to Cloud Computing

What is Cloud Computing?

Cloud computing is the delivery of various services over the internet. These services include storage, databases, servers, networking, software, analytics, and intelligence. Instead of owning their own computing infrastructure or data centers, companies can rent access to anything from applications to storage from a cloud service provider.

Key Characteristics of Cloud Computing

1. **On-Demand Self-Service**: Users can provision computing resources as needed automatically without requiring human interaction with each service provider.
2. **Broad Network Access**: Resources are available over the network and accessed through standard mechanisms that promote use by heterogeneous thin or thick client platforms (e.g., mobile phones, tablets, laptops, and workstations).
3. **Resource Pooling**: The provider's computing resources are pooled to serve multiple consumers using a multi-tenant model, with different physical and virtual resources dynamically assigned and reassigned according to consumer demand.
4. **Rapid Elasticity**: Capabilities can be elastically provisioned and released, in some cases automatically, to scale rapidly outward and inward commensurate with demand. To the consumer, the capabilities available for provisioning often appear to be unlimited and can be purchased in any quantity at any time.
5. **Measured Service**: Cloud systems automatically control and optimize resource use by leveraging a metering capability at some level of abstraction appropriate to the type of service (e.g., storage, processing, bandwidth, and active user accounts). Resource usage can be monitored, controlled, and reported, providing transparency for both the provider and consumer of the utilized service.

Types of Cloud Services

1. **Infrastructure as a Service (IaaS)**: Provides virtualized computing resources over the internet. IaaS platforms offer highly scalable resources that can be adjusted on-demand. Examples include Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP).

2. **Platform as a Service (PaaS)**: Provides a platform allowing customers to develop, run, and manage applications without dealing with the complexity of building and maintaining the underlying infrastructure. Examples include Google App Engine and Microsoft Azure.
3. **Software as a Service (SaaS)**: Delivers software applications over the internet, on a subscription basis. SaaS allows each user to access programs via the internet instead of having to install them on their computers. Examples include Google Workspace, Microsoft Office 365, and Salesforce.

Deployment Models

1. **Public Cloud**: The cloud infrastructure is provisioned for open use by the general public. It may be owned, managed, and operated by a business, academic, or government organization, or some combination of them. Examples include AWS, Azure, and GCP.
2. **Private Cloud**: The cloud infrastructure is provisioned for exclusive use by a single organization comprising multiple consumers (e.g., business units). It may be owned, managed, and operated by the organization, a third party, or some combination of them.
3. **Hybrid Cloud**: The cloud infrastructure is a composition of two or more distinct cloud infrastructures (private, community, or public) that remain unique entities but are bound together by standardized or proprietary technology that enables data and application portability.

Benefits of Cloud Computing

- **Cost Efficiency**: Reduces the capital expense of buying hardware and software and setting up and running on-site data centers.
- **Speed and Agility**: Vast amounts of computing resources can be provisioned in minutes, typically with just a few mouse clicks, giving businesses a lot of flexibility.
- **Scalability**: Allows businesses to scale up or scale down their IT resources as needed.
- **Performance**: Major cloud services run on a worldwide network of secure data centers, which are upgraded to the latest generation of fast and efficient computing hardware.

- **Security**: Many cloud providers offer a set of policies, technologies, and controls that strengthen security overall, helping protect data, apps, and infrastructure from potential threats.

****

Challenges in cloud computing:

1. Security: One of the most significant challenges of cloud computing is security. Businesses have to rely on cloud service providers to secure their data, which can be a challenge due to the vast amount of data being stored on the cloud.

2. Downtime: Cloud computing relies on internet connectivity, and businesses may experience downtime due to network or service provider issues. This can have a significant impact on business operations.

3. Limited Control: When businesses use cloud computing, they have limited control over the infrastructure, hardware, and software used to store and manage their data. This can make it difficult for businesses to customize their systems as per their needs.

4. Data Privacy: When businesses use cloud computing, they may have to share their data with third-party service providers, which can raise concerns about data privacy and ownership.

5. Dependence on Service Providers: Businesses relying on cloud computing services are dependent on the service providers for the availability and performance of their systems. This can make it difficult for businesses to switch service providers if the need arises.

Here are some of the key challenges in cloud computing:

1. Security and Privacy

- **Data Breaches**: Storing sensitive data in the cloud can make it a target for cyber-attacks. Ensuring robust encryption and access controls is crucial.
- **Compliance**: Meeting regulatory requirements (e.g., GDPR, HIPAA) can be challenging, especially when data is stored in multiple locations and jurisdictions.
- **Data Sovereignty**: Different countries have different laws regarding data storage and transfer, which can complicate compliance and legal issues.

2. Cost Management

- **Unexpected Costs**: While cloud services can be cost-effective, unexpected costs can arise from over-provisioning or underutilized resources.
- **Pricing Models**: The complexity of pricing models can make it difficult for organizations to predict and manage costs effectively.

3. Performance and Downtime

- **Latency**: Applications that require high-speed transactions may suffer from latency issues when using cloud services.
- **Downtime**: Even the most reliable cloud service providers experience outages, which can disrupt business operations.

4. Vendor Lock-In

- **Interoperability**: Moving applications and data between different cloud providers can be complex and costly due to differing platforms and APIs.
- **Dependency**: Relying heavily on a single provider can create risks if the provider changes its policies or goes out of business.

5. Data Transfer and Bandwidth

- **Bandwidth Costs**: Transferring large volumes of data to and from the cloud can incur significant costs.
- **Data Migration**: Migrating existing applications and data to the cloud can be complex, time-consuming, and costly.

6. Compliance and Legal Issues

- **Regulatory Compliance**: Ensuring that cloud operations comply with various regulations can be challenging, especially when regulations vary by country.
- **Legal Contracts**: Navigating the terms of service agreements and SLAs (Service Level Agreements) with cloud providers can be complex.

7. Integration with Existing Systems

- **Legacy Systems**: Integrating cloud services with existing on-premises systems and legacy applications can be challenging and may require significant re-engineering.
- **API Compatibility**: Ensuring that APIs used by different services and systems are compatible and work together seamlessly.

8. Skills and Expertise

- **Skill Gap**: Organizations may struggle to find employees with the necessary skills to manage and optimize cloud services.
- **Training**: Continuous training and development are needed to keep up with the rapidly evolving cloud technologies and best practices.

9. Governance and Control

- **Resource Management**: Effective governance policies are needed to manage resource allocation, usage, and cost.
- **Access Control**: Managing user access and permissions across different cloud services can be complex.

10. Environmental Impact

- **Energy Consumption**: Cloud data centers consume large amounts of energy, raising concerns about their environmental impact.
- **Sustainability**: Ensuring that cloud operations are sustainable and environmentally friendly is becoming increasingly important.

11. Technical Issues

- **Service Compatibility**: Ensuring that different cloud services and components work together without conflicts can be challenging.
- **Bug Management**: Identifying and resolving bugs or performance issues in a cloud environment can be more complex than in on-premises environments.

****

Cloud types

Cloud computing deployment models can be broadly categorized into four main types: Public Cloud, Private Cloud, Hybrid Cloud, and Multi-Cloud. Each type offers different benefits and is suited to different business needs and scenarios.

1. Public Cloud

Definition: The public cloud is a type of cloud computing in which a service provider makes resources, such as virtual machines (VMs), applications, or storage, available to the general public over the internet.

A sort of cloud computing architecture known as the "public cloud" allows for the delivery of cloud services via the internet by independent cloud providers. It is available to the general public, and anyone can use it by paying a fee or on a pay-per-use basis. Public clouds provide scalability, flexibility, and cost-effectiveness, which makes them ideal for small and medium-sized businesses. Public cloud services include SaaS, PaaS, and IaaS.

Examples:

- Amazon Web Services (AWS)
- Microsoft Azure
- Google Cloud Platform (GCP)
- IBM Cloud

Advantages:

- **Cost Efficiency**: Users pay only for the resources they use, with no need for upfront capital expenditures.
- **Scalability**: Resources can be scaled up or down quickly to meet changing demands.
- **Reliability**: Public cloud providers offer high levels of redundancy and disaster recovery.

Disadvantages:

- **Security Concerns**: Data is stored off-premises, which can raise concerns about data privacy and security.
- **Limited Customization**: Less control over the environment, which may limit customization options.

2. Private Cloud

Definition: A private cloud is a cloud computing environment dedicated to a single organization. It can be managed internally or by a third-party provider and can be hosted on-premises or off-site.

An organisation or a single entity owns and manages a private cloud, commonly referred to as an internal or corporate cloud. It is not accessible to the public, and its services are available only to authorized users within the organization. Private cloud services are designed to meet the specific requirements of an organization and provide complete control over data security and privacy. Private clouds are ideal for organizations that require a high level of security, compliance, and customization.

Examples:

- VMware vCloud
- Microsoft Azure Stack
- OpenStack

Advantages:

- **Enhanced Security**: Greater control over data and infrastructure, which can lead to better security and privacy.
- **Customization**: Highly customizable to meet specific business needs.
- **Compliance**: Easier to meet regulatory and compliance requirements.

Disadvantages:

- **Higher Costs**: Requires significant investment in hardware, software, and expertise.

- **Scalability Limitations**: Scaling may be limited by the capacity of the internal infrastructure.

3. Hybrid Cloud

Definition: A hybrid cloud is a computing environment that combines a public cloud and a private cloud, allowing data and applications to be shared between them. This model provides greater flexibility and more deployment options.

Public and private cloud deployment models are combined to create a hybrid cloud. It gives businesses the freedom to select the optimum cloud deployment approach for their particular requirements. With a hybrid cloud, businesses can use private cloud services for sensitive data and public cloud services for less-sensitive data. For businesses that need the advantages of both public and private cloud deployment options, hybrid clouds are the best option.

Examples:

- Microsoft Azure Hybrid
- AWS Outposts
- Google Anthos

Advantages:

- **Flexibility**: Ability to use both on-premises and public cloud resources as needed.
- **Cost Management**: Allows organizations to optimize costs by running workloads in the most cost-effective environment.
- **Improved Security**: Sensitive data can be kept on-premises while leveraging public cloud resources for less critical workloads.

Disadvantages:

- **Complexity**: Managing and integrating multiple environments can be complex and require specialized skills.
- **Security Risks**: Data transfer between environments can introduce security risks if not properly managed.

4. Multi-Cloud

Definition: Multi-cloud refers to the use of multiple cloud computing services from different providers in a single heterogeneous architecture. This approach avoids reliance on any single cloud provider.

Examples:

- Using AWS for data storage, Azure for application hosting, and GCP for machine learning.

Advantages:

- **Avoid Vendor Lock-In**: Reduces dependency on a single cloud provider.
- **Optimized Performance**: Ability to choose the best cloud service for each specific workload.
- **Increased Reliability**: Redundancy across multiple providers can enhance reliability and uptime.

Disadvantages:

- **Management Complexity**: Managing multiple cloud services and ensuring interoperability can be challenging.
- **Higher Costs**: Potentially higher costs due to the need for multiple management tools and platforms.

Comparison

Deployment Model	Accessibility	Security	Customization	Cost
Public Cloud	Available to the general public	Shared security responsibility	Limited customization	Pay-per-use
Private Cloud	Restricted to authorized users	Complete control over security	High customization	High initial investment
Hybrid Cloud	use of both public and private clouds	Flexible security	High customization	Pay-per-use and initial investment
Community Cloud	Shared by multiple organizations	Shared security responsibility	Customized to the community's needs	Shared cost

Table 1.3 Cloud Deployment Models

****

CLOUD COMPUTING ARCHITECTURE

Cloud Computing means storing and accessing the data and programs on remote servers that are hosted on the internet instead of the computer's hard drive or local server. Cloud computing is also referred to as Internet-based computing, it is a technology where the resource is provided as a service through the Internet to the user. The data that is stored can be files, images, documents, or any other storable document. Transparency, scalability, security and intelligent monitoring are some of the most important constraints which every cloud infrastructure should experience. Current research on other important constraints is helping cloud computing system to come up with new features and strategies with a great capability of providing more advanced cloud solutions.

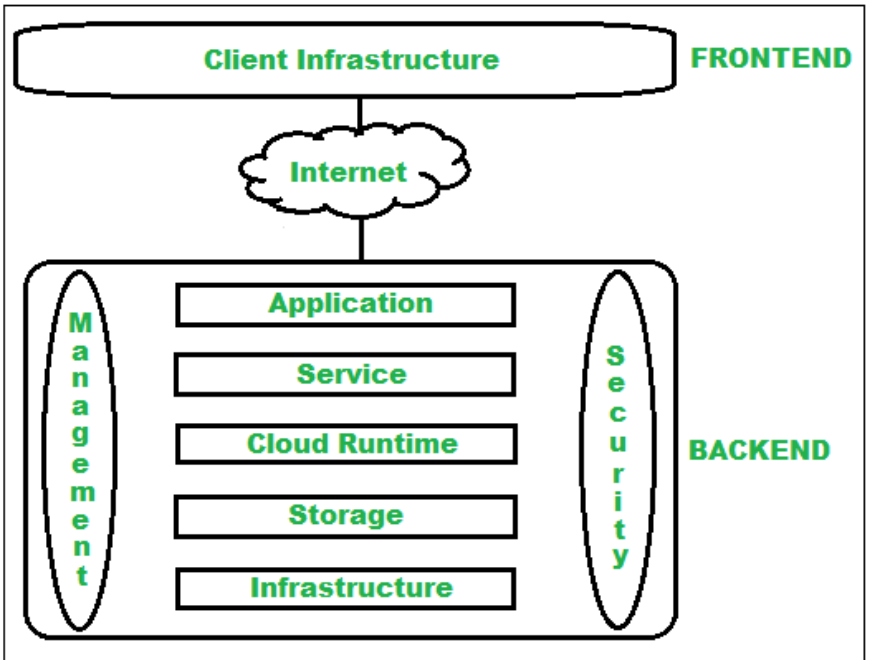
Cloud Computing Architecture

Architecture of cloud computing is the combination of both SOA (Service Oriented Architecture) and EDA (Event Driven Architecture). Client infrastructure, application, service, runtime cloud, storage, infrastructure, management and security all these are the components of cloud computing architecture.

The cloud architecture is divided into 2 parts, i.e.

1. Frontend
2. Backend

The below figure represents an internal architectural view of cloud computing.



Architecture of Cloud Computing

Cloud architecture refers to the components and subcomponents required for cloud computing, which typically includes a front-end platform, back-end platforms, a cloud-based delivery, and a network. It involves various elements such as hardware, virtualization, storage, and network resources that work together to deliver cloud services. Here is a detailed overview of the components of cloud architecture:

1. Front-End Platform

Frontend of the cloud architecture refers to the client side of cloud computing system. Means it contains all the user interfaces and applications which are used by the client to access the cloud computing services/resources. For example, use of a web browser to access the cloud platform.

Components:

- **Client Devices**: Devices such as computers, tablets, and smartphones that users use to interact with cloud services.
- **User Interfaces**: Interfaces like web browsers and mobile apps through which users access cloud services.

2. Back-End Platform

Backend refers to the cloud itself which is used by the service provider. It contains the resources as well as manages the resources and provides security mechanisms. Along with this, it includes huge storage, virtual applications, virtual machines, traffic control mechanisms, deployment models, etc.

Components:

- **Servers**: The physical or virtual machines that host applications and services.
- **Storage**: Databases and file storage systems where data is stored.
- **Networking**: The infrastructure that supports data transfer and connectivity between servers, storage, and users.

3. Cloud-Based Delivery

Components:

- **Software as a Service (SaaS)**: Delivers software applications over the internet on a subscription basis (e.g., Google Workspace, Microsoft Office 365).
- **Platform as a Service (PaaS)**: Provides a platform allowing customers to develop, run, and manage applications without dealing with the underlying infrastructure (e.g., Google App Engine, Microsoft Azure).
- **Infrastructure as a Service (IaaS)**: Offers virtualized computing resources over the internet (e.g., Amazon Web Services, Microsoft Azure).

4. Network

Components:

- **Internet**: The global system of interconnected computer networks that enables cloud services to be accessed remotely.
- **Intranets**: Private networks used within an organization to connect various components of the cloud architecture securely.

5. Middleware

Middleware is software that lies between the client-side and server-side of an application, providing services such as messaging, authentication, API management, and monitoring.

6. Virtualization

Virtualization is a key component of cloud architecture, enabling the creation of virtual instances of servers, storage, and networks, thus allowing for greater flexibility and efficiency.

7. Security

Security in backend refers to implementation of different security mechanisms in the backend for secure cloud resources, systems, files, and infrastructure to end-users.

Security components are integral to cloud architecture, ensuring data protection, access control, and compliance with regulations. This includes:

- **Firewalls**: Protecting against unauthorized access and threats.

- **Encryption**: Securing data both at rest and in transit.

Components of Cloud Computing Architecture

Some of the important components of Cloud Computing architecture that we will be looking into are as follows:

- Hypervisor
- Management Software
- Deployment Software
- Network
- Cloud Server
- Cloud Storage



Following are the components of Cloud Computing Architecture

Client Infrastructure – Client Infrastructure is a part of the frontend component. It contains the applications and user interfaces which are required to access the cloud

platform. In other words, it provides a GUI(Graphical User Interface) to interact with the cloud.

Application : Application is a part of backend component that refers to a software or platform to which client accesses. Means it provides the service in backend as per the client requirement.

Service: Service in backend refers to the major three types of cloud based services like SaaS, PaaS and IaaS. Also manages which type of service the user accesses.

Runtime Cloud: Runtime cloud in backend provides the execution and Runtime platform/environment to the Virtual machine.

Storage: Storage in backend provides flexible and scalable storage service and management of stored data.

Infrastructure: Cloud Infrastructure in backend refers to the hardware and software components of cloud like it includes servers, storage, network devices, virtualization software etc.

Management: Management in backend refers to management of backend components like application, service, runtime cloud, storage, infrastructure, and other security mechanisms etc.

Its task is to allot specific resources to a specific task, it simultaneously performs various functions of the cloud environment

It helps in the management of components like application, task, service, security, data storage, and cloud infrastructure

In simple terms, it establishes coordination among the cloud resources

Security: Security is an integral part of back-end cloud infrastructure

- It provides secure cloud resources, systems, files, and infrastructure to end-users
- Also, it implements security management to the cloud server with virtual firewalls which results in preventing data loss

Storage

- It stores and maintains data like files, videos, documents, etc. over the internet
- Some of the popular examples of storage services are below:
    - Amazon S3
    - Oracle Cloud-Storage
    - Microsoft Azure Storage
- Its capacity varies depending upon the service providers available in the market

10. Service

- Service is an essential component in cloud architecture
- Its responsibility is to provide utility in the architecture
- In a Cloud, few widely used services among the end-users are storage application development environments and web services

Internet: Internet connection acts as the medium or a bridge between frontend and backend and establishes the interaction and communication between frontend and backend.
Database: Database in backend refers to provide database for storing structured data, such as SQL and NOSQL databases. Example of Databases services include Amazon RDS, Microsoft Azure SQL database and Google CLoud SQL.
Networking: Networking in backend services that provide networking infrastructure for application in the cloud, such as load balancing, DNS and virtual private networks.
Analytics: Analytics in backend service that provides analytics capabilities for data in the cloud, such as warehousing, business intelligence and machine learning.

Benefits of Cloud Computing Architecture

- Makes overall cloud computing system simpler.
- Improves data processing requirements.
- Helps in providing high security.
- Makes it more modularized.
- Results in better disaster recovery.
- Gives good user accessibility.
- Reduces IT operating costs.
- Provides high level reliability.
- Scalability.

The cloud computing architecture is designed in such a way that:

- It solves latency issues and improves data processing requirements
- It reduces IT operating costs and gives good accessibility to access data and digital tools
- It helps businesses to easily scale up and scale down their cloud resources
- It has a flexibility feature which gives businesses a competitive advantage
- It results in better disaster recovery and provides high security

- It automatically updates its services
- It encourages remote working and promotes team collaboration

Cloud Computing architecture provides a structural framework for designing, implementing and managing cloud-based solutions. Cloud Computing Architecture provides benefits like scalability, flexibility, and cost-effectiveness. It also solve related to security, reliability, and performance.

****

CLOUD SERVICES

Cloud services encompass a wide range of services delivered over the internet, which can be broadly categorized into three main types: Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS). Each of these categories offers different levels of control, flexibility, and management responsibilities to the user.

1. Infrastructure as a Service (IaaS)

Definition: IaaS provides virtualized computing resources over the internet. It offers the most basic cloud service model, giving users direct access to virtualized hardware such as virtual machines (VMs), storage, and networks.

Infrastructure as a Service (IaaS): a cloud service delivery paradigm that gives consumers pay-per-use access to virtualized computer resources like servers, storage, and networking parts. The virtualized infrastructure that IaaS consumers normally control can be changed, but are responsible for managing and maintaining their own software applications and data.

Infrastructure as a Service (IaaS) provides customers with access to a variety of computer resources, including virtual machines, storage, and networking. Users of IaaS are in charge of administering and maintaining the operating system, middleware, and applications. IaaS providers like Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform are a few examples.

Key Features:

- **Compute**: Virtual machines with various configurations of CPU, memory, and storage.
- **Storage**: Scalable storage options including block storage, file storage, and object storage.
- **Networking**: Virtual networks, load balancers, and IP addresses.
- **Flexibility**: Users have full control over the infrastructure and can run any software or operating system.

Examples:

- **Amazon Web Services (AWS)**: EC2, S3, EBS
- **Microsoft Azure**: Virtual Machines, Blob Storage, Virtual Network
- **Google Cloud Platform (GCP)**: Compute Engine, Cloud Storage, VPC

2. Platform as a Service (PaaS)

Definition: PaaS provides a platform allowing customers to develop, run, and manage applications without dealing with the complexity of building and maintaining the underlying infrastructure.

Platform as a Service (PaaS): a cloud service paradigm that offers users a network-based environment for creating, deploying, and managing software applications. PaaS users typically have access to a pre-configured platform that includes tools, libraries, and other resources to facilitate software development and deployment, while the operating system and supporting infrastructure are taken care of by the service provider.

Platform as a Service (PaaS) is a cloud service paradigm that offers users a full application development and deployment environment. For developers to design, test, and deploy 8 applications fast and efficiently, PaaS providers offer a variety of pre-built tools and services, including databases, middleware, and development frameworks. PaaS services like Heroku, Google App Engine, and Microsoft Azure are examples. Users can access software programmes through the internet thanks to the cloud service concept known as "Software as a Service" (SaaS). SaaS providers are in charge of managing the infrastructure, middleware, and applications for the full software package. Only a web browser or a mobile application is

required for users to access the software. Salesforce, Dropbox, and Microsoft Office 365 are just a few examples of SaaS vendors.

Key Features:

- **Development Tools**: Integrated development environments (IDEs), version control, and build automation tools.
- **Middleware**: Application servers, databases, and message queues.
- **Database Management**: Managed database services for SQL and NoSQL databases.
- **Application Hosting**: Environments for deploying and scaling web applications and APIs.

Examples:

- **Google App Engine**: Fully managed platform for building and deploying applications.
- **Microsoft Azure App Service**: Platform for building, deploying, and scaling web apps.
- **Heroku**: Cloud platform that supports several programming languages and is used for deploying applications quickly.

3. Software as a Service (SaaS)

Definition: SaaS delivers software applications over the internet, on a subscription basis. Users can access these applications via a web browser or API, without needing to install or manage the underlying infrastructure or software.

Software as a Service (SaaS): a cloud service paradigm that enables customers to connect to and utilise software hosted and controlled by a service provider over a network. SaaS users typically pay a subscription fee and have little control over the underlying 4 infrastructure or software code, but benefit from the ease of use, scalability, and reduced management overhead provided by the service provider.

Key Features:

- **Accessibility**: Applications are accessible from any device with an internet connection.
- **Maintenance**: The service provider handles all maintenance, updates, and security.
- **Scalability**: Users can easily scale their usage based on their needs.
- **Subscription-Based**: Typically offered on a pay-as-you-go or subscription basis.

Examples:

- **Google Workspace**: Suite of productivity and collaboration tools including Gmail, Docs, and Drive.
- **Microsoft Office 365**: Cloud-based versions of Microsoft Office applications.
- **Salesforce**: Customer relationship management (CRM) software.

Other Cloud Service Models

4. Function as a Service (FaaS)

Definition: FaaS, also known as serverless computing, allows developers to execute code in response to events without managing servers. The cloud provider handles the infrastructure, scaling, and execution.

Key Features:

- **Event-Driven**: Code is executed in response to events such as HTTP requests or file uploads.
- **Automatic Scaling**: Automatically scales based on the number of incoming requests.
- **Cost-Efficiency**: Users are billed only for the actual execution time of the code.

Examples:

- **AWS Lambda**: Service for running code in response to events.
- **Google Cloud Functions**: Serverless environment to build and connect cloud services.
- **Azure Functions**: Event-driven serverless compute platform.

5. Storage as a Service (STaaS)

Definition: STaaS provides users with scalable and on-demand storage resources.

Key Features:

- **Scalability**: Storage capacity can be easily scaled up or down based on requirements.
- **Accessibility**: Data can be accessed from anywhere with an internet connection.
- **Durability and Redundancy**: Ensures high availability and data protection through replication and backups.

Examples:

- **Amazon S3**: Object storage service with high availability and durability.
- **Google Cloud Storage**: Scalable object storage for any amount of data.
- **Azure Blob Storage**: Storage solution for large amounts of unstructured data.

6. Database as a Service (DBaaS)

Definition: DBaaS provides managed database services, allowing users to access and use databases without managing the underlying hardware or software.

Key Features:

- **Managed Service**: Provider handles database management tasks such as backups, patching, and scaling.
- **Scalability**: Databases can be scaled horizontally and vertically to meet application demands.
- **Security**: Built-in security features including encryption and access controls.

Examples:

- **Amazon RDS**: Managed relational database service.
- **Google Cloud SQL**: Managed database service for MySQL, PostgreSQL, and SQL Server.
- **Azure SQL Database**: Managed relational database service with built-in intelligence.

Service Model	Description	Examples
IaaS	Provides users with access to computing resources such as virtual machines, storage, and networking. Users are responsible for managing the operating system, middleware, and applications.	Amazon Web Services, Microsoft Azure, Google Cloud Platform
PaaS	Provides users with a complete development and deployment environment for their applications. PaaS providers offer a range of pre-built tools and services to help developers build, test, and deploy applications.	Heroku, Google App Engine, Microsoft Azure
SaaS	Provides users with access to software applications over the internet. SaaS providers manage the entire software application, including the infrastructure, middleware, and applications.	Salesforce, Dropbox, Microsoft Office 365

****

Communication Protocol and application

Communication protocols are essential for enabling seamless interactions between different components in cloud computing. These protocols ensure data is transmitted securely, reliably, and efficiently across networks and between cloud services. Here's an overview of some key communication protocols and their applications in cloud computing:

1. HTTP/HTTPS

Definition: HyperText Transfer Protocol (HTTP) and its secure version, HTTP Secure (HTTPS), are foundational protocols for data communication on the web.

Applications:

- **Web Services**: HTTP/HTTPS is widely used for RESTful APIs, allowing web services to communicate over the internet.
- **Web Applications**: Used to serve web pages and applications from cloud servers to users' browsers.
- **Data Transfer**: Enables secure data transfer between clients and servers using encryption in HTTPS.

2. TCP/IP

Definition: Transmission Control Protocol/Internet Protocol (TCP/IP) is the fundamental suite of protocols for communication over the internet.

Applications:

- **Reliable Data Transfer**: Ensures reliable, ordered, and error-checked delivery of data between applications.
- **Internet Connectivity**: Facilitates communication between cloud services and users over the internet.
- **Network Communication**: Underpins most other protocols used in cloud environments.

3. WebSocket

Definition: WebSocket is a protocol providing full-duplex communication channels over a single TCP connection.

Applications:

- **Real-Time Applications**: Used in applications requiring real-time updates, such as live chat, gaming, and financial trading platforms.
- **Interactive Web Applications**: Enables real-time interactions in web applications, enhancing user experience.

4. MQTT

Definition: Message Queuing Telemetry Transport (MQTT) is a lightweight, publish-subscribe network protocol for transporting messages between devices.

Applications:

- **IoT**: Widely used in Internet of Things (IoT) applications for its efficiency and low overhead.
- **Remote Monitoring**: Ideal for scenarios requiring reliable, low-bandwidth communication.

5. AMQP

Definition: Advanced Message Queuing Protocol (AMQP) is an open standard application layer protocol for message-oriented middleware.

Applications:

- **Message Brokers**: Used by message brokers like RabbitMQ to facilitate communication between distributed systems.
- **Enterprise Messaging**: Ensures reliable and secure message delivery in enterprise applications.

6. SOAP

Definition: Simple Object Access Protocol (SOAP) is a protocol for exchanging structured information in the implementation of web services.

Applications:

- **Web Services**: Used for invoking web service methods and exchanging structured data.
- **Enterprise Integration**: Facilitates communication between different software components in enterprise environments.

7. gRPC

Definition: gRPC is a high-performance, open-source remote procedure call (RPC) framework developed by Google.

Applications:

- **Microservices**: Enables efficient communication between microservices in cloud-native architectures.
- **Cross-Language Communication**: Supports multiple programming languages, making it suitable for heterogeneous environments.

8. DNS

Definition: Domain Name System (DNS) is a hierarchical system for naming resources on the internet.

Applications:

- **Service Discovery**: Used for discovering and routing to services in cloud environments.
- **Load Balancing**: Distributes traffic among multiple servers to ensure availability and performance.

Security Protocols

9. SSL/TLS

Definition: Secure Sockets Layer (SSL) and Transport Layer Security (TLS) are cryptographic protocols designed to provide secure communication over a computer network.

Applications:

- **Encryption**: Encrypts data in transit to protect against eavesdropping and tampering.
- **Authentication**: Verifies the identity of communicating parties to prevent man-in-the-middle attacks.

10. IPsec

Definition: Internet Protocol Security (IPsec) is a suite of protocols for securing IP communications by authenticating and encrypting each IP packet.

Applications:

- **VPNs**: Commonly used in virtual private networks (VPNs) to secure remote access to cloud resources.
- **Secure Communication**: Ensures secure data transfer across potentially insecure networks.

Applications of Communication Protocols in Cloud Computing

1. **API Management**:
    - Protocols like HTTP/HTTPS, REST, and gRPC are used to design and manage APIs, enabling different cloud services and applications to interact seamlessly.
2. **Microservices Communication**:
    - gRPC, WebSocket, and RESTful APIs facilitate communication between microservices, ensuring low-latency and high-performance interactions.
3. **Data Streaming and Messaging**:
    - Protocols like MQTT and AMQP are used for real-time data streaming and messaging between distributed systems, particularly in IoT and real-time analytics.
4. **Secure Access and Data Protection**:
    - SSL/TLS and IPsec protocols are essential for securing data in transit and ensuring secure access to cloud resources.
5. **Service Discovery and Load Balancing**:
    - DNS and other service discovery protocols help in load balancing and routing traffic to ensure high availability and reliability of cloud services.

****

Adafruit –Communicating with cloud

Adafruit is a well-known company in the maker community, providing hardware, software, and tutorials for DIY electronics projects. One of the areas Adafruit focuses on is enabling Internet of Things (IoT) projects to communicate with cloud services. Here's an overview of how Adafruit devices can communicate with the cloud and the tools they provide to facilitate this communication.

Adafruit IO

Adafruit IO is Adafruit's cloud service designed specifically for IoT projects. It enables devices to send data to the cloud and receive data from the cloud, making it easier to create connected applications.

Key Features:

- **Data Feeds**: Streams of data (like temperature readings, button presses, etc.) that you can create, update, and view.
- **Dashboards**: Visual interfaces to interact with your data feeds using widgets like charts, sliders, and buttons.
- **Triggers**: Automation rules that can trigger actions when data in a feed meets certain conditions.
- **API**: REST API and MQTT API for sending and receiving data from Adafruit IO.

Communication Protocols

Adafruit devices primarily use two protocols to communicate with Adafruit IO and other cloud services:

1. **MQTT (Message Queuing Telemetry Transport)**:
    - A lightweight, publish-subscribe network protocol ideal for IoT applications.
    - Allows devices to publish data to "topics" and subscribe to topics to receive data.
    - Adafruit IO supports MQTT, making it straightforward to integrate Adafruit hardware with the cloud.
2. **HTTP/HTTPS**:

- Standard web protocols used for sending data to web servers and APIs.
- Adafruit IO's REST API uses HTTP/HTTPS, enabling devices to send data directly to the cloud using simple HTTP requests.

Hardware Components

Adafruit offers several microcontrollers and development boards that are well-suited for IoT projects and can communicate with the cloud:

1. **Adafruit Feather**:
   - A versatile family of development boards designed to be lightweight and powerful.
   - Variants include built-in Wi-Fi (e.g., Feather HUZZAH with ESP8266, Feather M0 WiFi) and cellular connectivity (e.g., Feather 32u4 FONA).
2. **Adafruit PyPortal**:
   - An IoT display with a touchscreen that can connect to Wi-Fi and communicate with cloud services.
   - Ideal for creating custom dashboards and interfaces that interact with cloud data.
3. **Adafruit ESP8266/ESP32**:
   - Low-cost Wi-Fi modules that can be used to connect your projects to the internet.
   - The ESP8266 and ESP32 microcontrollers are widely supported and have extensive libraries for cloud communication.

Software Libraries

Adafruit provides several software libraries to help you connect your hardware to the cloud:

1. **Adafruit IO Arduino Library**:
   - A library for Arduino-compatible boards to communicate with Adafruit IO using MQTT and REST APIs.
   - Simplifies the process of sending data to and receiving data from Adafruit IO.
2. **Adafruit MQTT Library**:

    - A generic MQTT library for Arduino that supports connecting to any MQTT broker.
    - Useful for projects that need to communicate with other MQTT-based cloud services.
3. **CircuitPython**:
    - Adafruit's open-source derivative of MicroPython, designed for small microcontrollers.
    - Supports networking modules and has libraries for HTTP and MQTT communication.

Example Project: Sending Sensor Data to Adafruit IO

Here's a basic example of how to use an Adafruit Feather board to send temperature data to Adafruit IO using MQTT.

Hardware Setup:

- Adafruit Feather HUZZAH with ESP8266
- DHT22 temperature and humidity sensor
- Breadboard and jumper wires

Software Setup:

1. **Install Arduino IDE**: Ensure you have the Arduino IDE installed and set up for Adafruit Feather boards.
2. **Install Libraries**:
    - Adafruit MQTT Library
    - Adafruit IO Arduino Library
    - DHT Sensor Library

Example Code:

```
#include <Adafruit_MQTT.h>

#include <Adafruit_MQTT_Client.h>
```

```
#include <Adafruit_Sensor.h>

#include <DHT.h>

#include <DHT_U.h>

#include <ESP8266WiFi.h>


// WiFi credentials

#define WLAN_SSID       "your_SSID"

#define WLAN_PASS       "your_PASSWORD"


// Adafruit IO credentials

#define AIO_SERVER      "io.adafruit.com"

#define AIO_SERVERPORT  1883                   // Use 8883 for SSL

#define AIO_USERNAME    "your_AIO_USERNAME"

#define AIO_KEY         "your_AIO_KEY"


// Create WiFi client and MQTT client instances

WiFiClient client;

Adafruit_MQTT_Client mqtt(&client, AIO_SERVER, AIO_SERVERPORT, AIO_USERNAME, AIO_KEY);


// Create MQTT feed instances

Adafruit_MQTT_Publish temperatureFeed = Adafruit_MQTT_Publish(&mqtt, AIO_USERNAME "/feeds/temperature");
```

```
// DHT sensor setup
#define DHTPIN          2
#define DHTTYPE         DHT22
DHT_Unified dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(115200);
  delay(10);

  // Connect to WiFi
  Serial.print("Connecting to ");
  Serial.println(WLAN_SSID);
  WiFi.begin(WLAN_SSID, WLAN_PASS);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("WiFi connected");

  // Initialize DHT sensor
  dht.begin();
}
```

```
void loop() {
  // Ensure MQTT connection
  if (mqtt.connected() == false) {
    reconnect();
  }

  // Read temperature
  sensors_event_t event;
  dht.temperature().getEvent(&event);
  if (!isnan(event.temperature)) {
    // Publish temperature data
    if (!temperatureFeed.publish(event.temperature)) {
      Serial.println(F("Failed to publish temperature"));
    } else {
      Serial.println(F("Temperature published!"));
    }
  }

  // Wait before next reading
  delay(2000);
}
```

```
void reconnect() {
  while (mqtt.connected() == false) {
    Serial.print("Connecting to MQTT... ");
    if (mqtt.connect()) {
      Serial.println("connected");
    } else {
      Serial.print("failed, rc=");
      Serial.print(mqtt.connectErrorString(mqtt.connectError()));
      Serial.println(" retrying in 5 seconds");
      delay(5000);
    }
  }
}
```